

Reviewer Pack — Hypercontractive Constants of Tensor Powers and Max-CUT Appr...

Entry adcdc810fef8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 18:13:06 UTC

Hypercontractive Constants of Tensor Powers and Max-CUT Approximation

Entry ID: adcdc810fef8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 18:13:06 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic combinatorics over the tropical semiring **Field B** (complexity object): Sum-of-squares lower bound for max-CUT

Statement:

[‘For any degree- d SOS approximation algorithm for max-CUT, the probability that a tensor power of the input tensor with respect to the Fourier-analytic norm exceeds its hypercontractive constant is at most $e^{-(n/2d)}$.’, ‘This bound holds for all $n \leq 40$ and all $d \geq 2$.’, ‘For any counterexample violating this bound, there exists an instance where the SOS approximation algorithm fails to achieve a better than 0.879 approximation ratio.’]

Rationale (proposer’s reasoning):

[‘The hypercontractive constants of tensor powers are known to be closely related to the spectral properties of random matrices, which have been used in bounding communication complexity and influence functions.’, ‘By connecting this concept with the SOS hierarchy for max-CUT, we aim to provide a new perspective on the hardness of approximating max-CUT that could potentially lead to improved understanding or constructions of lower bounds.’, ‘This conjecture builds upon existing results in Fourier-analytic combinatorics and aims to apply them to a different area of complexity theory.’]

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 471bc653d0b356b6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given $n \leq 40$ and $d \geq 2$, if the SOS approximation algorithm’s tensor power norms exceed their hypercontractive constants with probability $< e^{-(n/2d)}$, or any seed produces a metric > 0.879 , then the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('tensor power' AND Fourier-analytic norm) AND ('hypercontractive constant' OR 'SOS approximation')-max-CUT AND ('sum-of-squares lower bound' OR 'tropical semiring')-Fourier-analytic combinatorics AND ('SOS approximation algorithm' OR 'max-CUT approximation ratio')

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 30 # Fixed size for simplicity
    d = 2 # Minimum degree for SOS approximation

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0]*p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
```

```

        C[i][j] += A[i][k] * B[k][j]

    return C

def frobenius_norm(A):
    norm = 0
    for row in A:
        for elem in row:
            norm += elem ** 2
    return math.sqrt(norm)

def hypercontractive_constant(n, d):
    return (1 - 1 / (d * n)) ** (n // 2)

tensor_power = [[random.random() for _ in range(n)] for _ in range(n)]
norm = frobenius_norm(tensor_power)
hyper_const = hypercontractive_constant(n, d)

if norm > hyper_const:
    return {
        "metric_name": "norm_exceeds_hyper_const",
        "metric_value": norm,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Norm {norm} exceeds hypercontractive constant {hyper_const}"
    }

return {
    "metric_name": "norm_exceeds_hyper_const",
    "metric_value": norm,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 50))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"norm_exceeds_hyper_const\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_name': 'norm_exceeds_hyper_const', 'metric_value': 17.35683033609339, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.093249894899063, 'instances_t
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.4765127177598, 'instances_tes
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.30285904651998, 'instances_te
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 16.854907159849372, 'instances_t
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.37771771513615, 'instances_te
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.61718285095232, 'instances_te
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.148879535083957, 'instances_t
TRIAL: {'metric_name': 'norm_exceeds_hyper_const', 'metric_value': 17.252851118222637, 'instances_t

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only considered $n \leq 40$, which is a very small range for empirical verification of such a conjecture. The metric does not scale trivially with n , but the size of n tested here may be too small to draw a definitive conclusion.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for at least one seed, the tensor power norm exceeds the hypercontractive constant, violating the conjecture’s bound. | next: Further investigation is needed to find an instance where the SOS approximation algorithm fails to achieve a better than 0.879 approximation ratio.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14881
2	preregistration	ollama_remote	glm4:latest	0	0	9639
3	novelty	ollama_remote	glm4:latest	0	0	9050
4	novelty	ollama_remote	glm4:latest	0	0	8789
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13370
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11222
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11371
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9443
9	critic	ollama_remote	glm4:latest	0	0	12395
10	judge	ollama_remote	glm4:latest	0	0	8963

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109124 ms total latency. Provider mix: {'ollama remote': 10}

(full prompt+response transcripts available in [research/audit/adcdc810fef8.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/adcdc810fef8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/adcdc810fef8.tar.gz` (if generated)